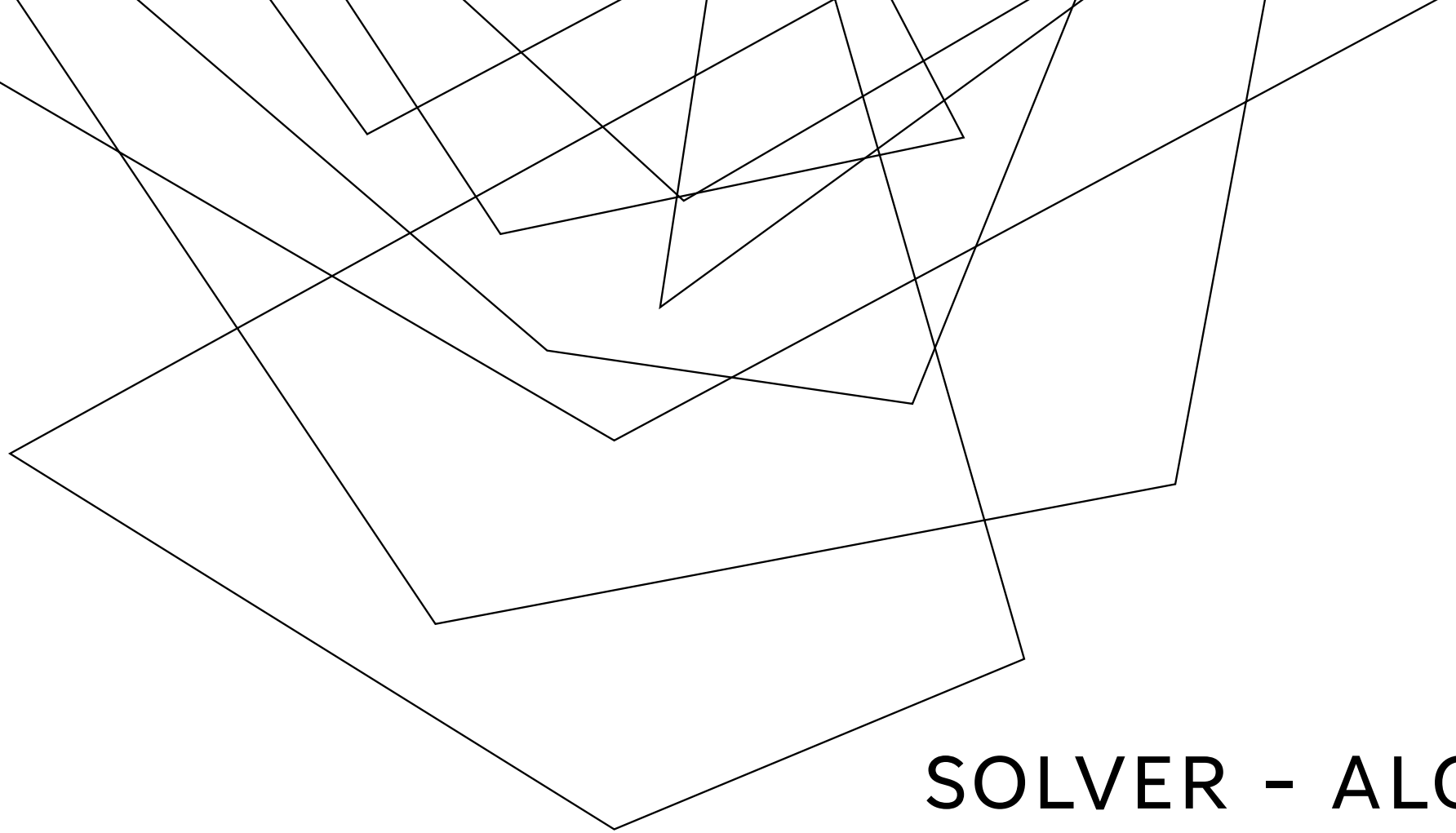


SOLVER - ALGORITHMEN

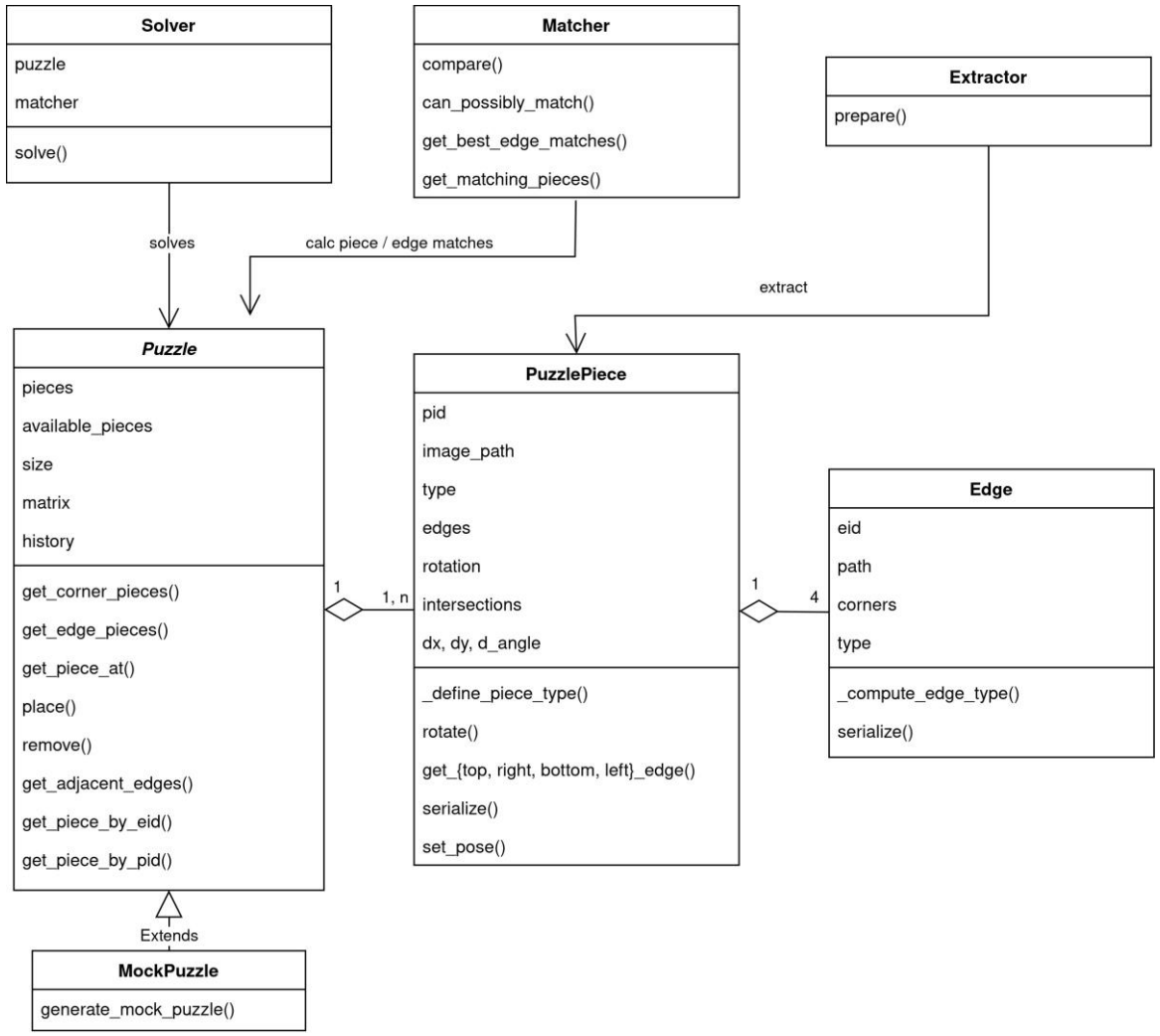
ROBOTIK PROJEKTSTUDIUM
PROF DR. FISCHER - SS24

GLIEDERUNG

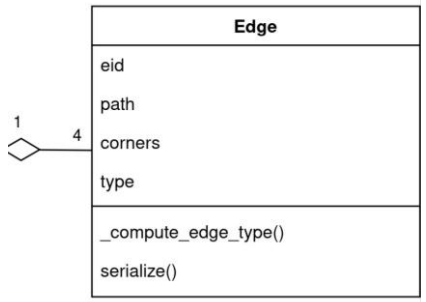
- Klassenstruktur
- Rekursive Algorithmen
  - Tiefensuche
  - Best-First-Algorithmus

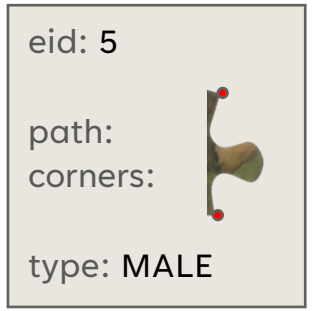
SOLVER / EINLEITUNG / KLASSENSTRUKTUR

SOLVER / EINLEITUNG / KLASSENSTRUKTUR

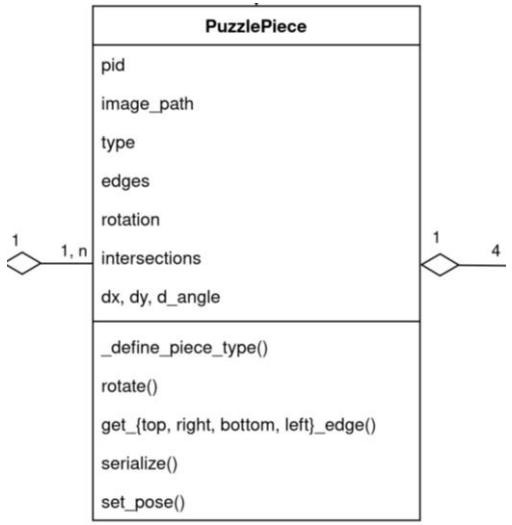


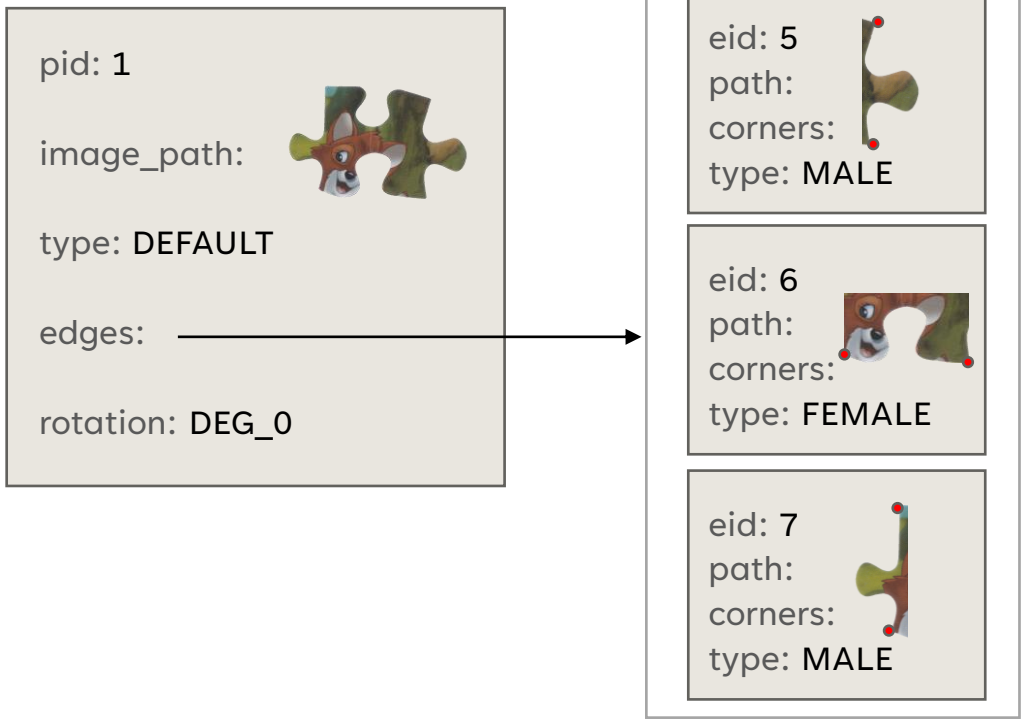
SOLVER / EINLEITUNG / KLASSENSTRUKTUR



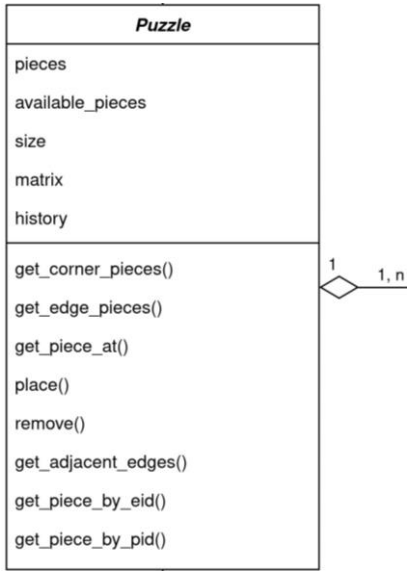


SOLVER / EINLEITUNG / KLASSENSTRUKTUR





SOLVER / EINLEITUNG / KLASSENSTRUKTUR

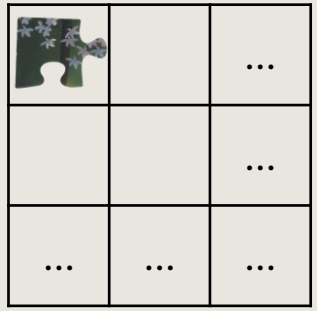


pieces: []

available_pieces: []

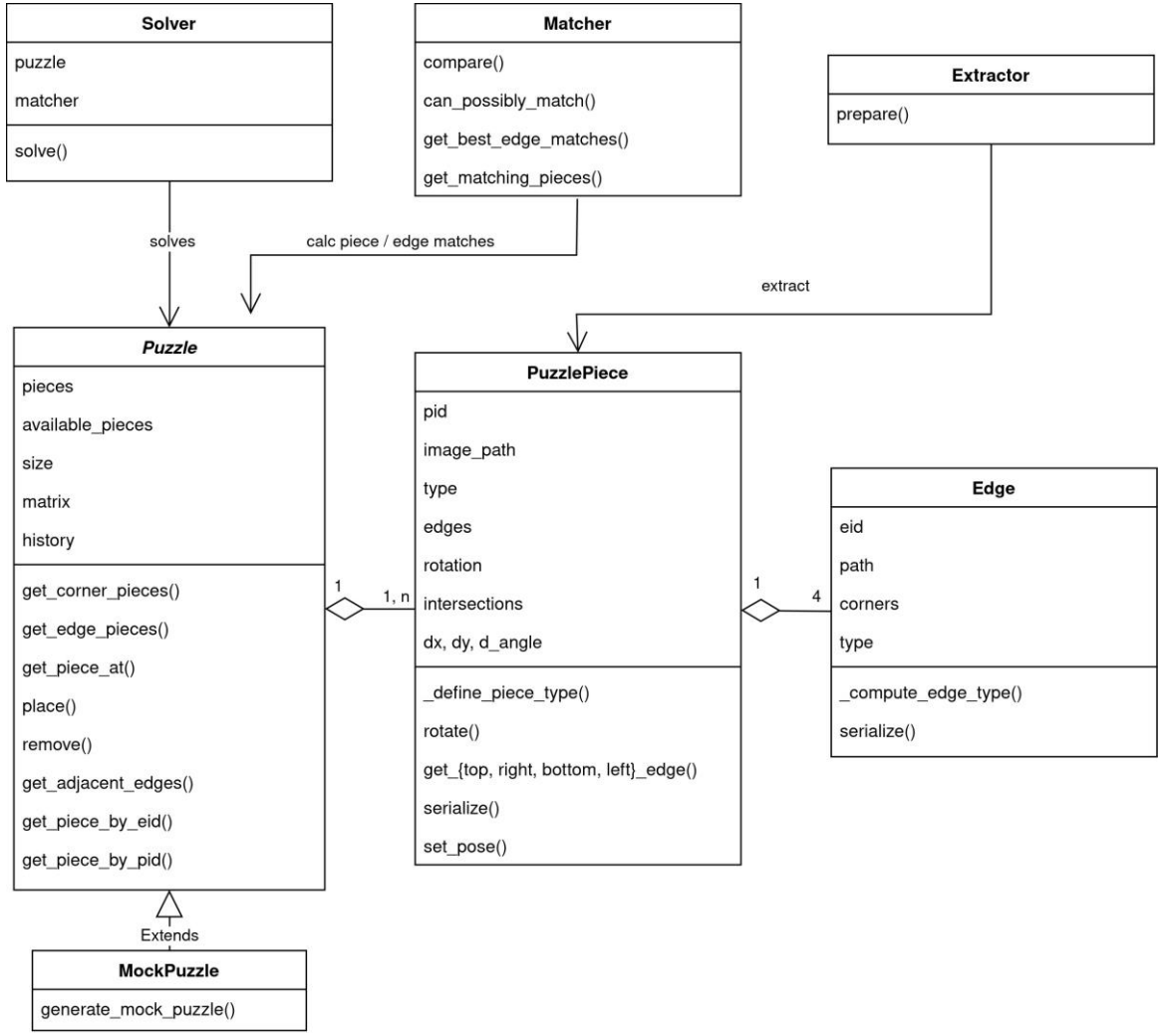
size: (7, 7)

matrix:

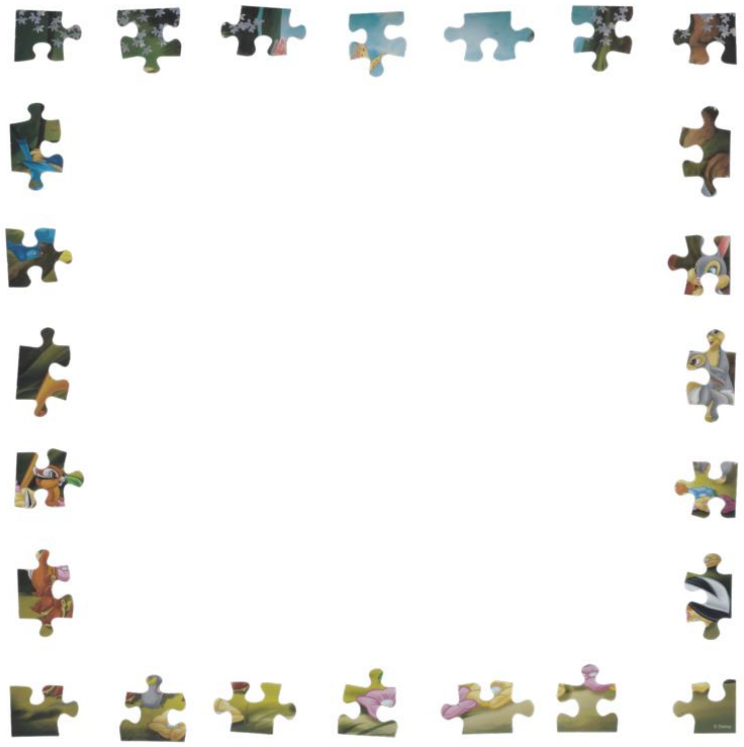


history: [(0,0), ..]

SOLVER / EINLEITUNG / KLASSENSTRUKTUR



SOLVER / ALGORITHMEN / TIEFENSUCHE

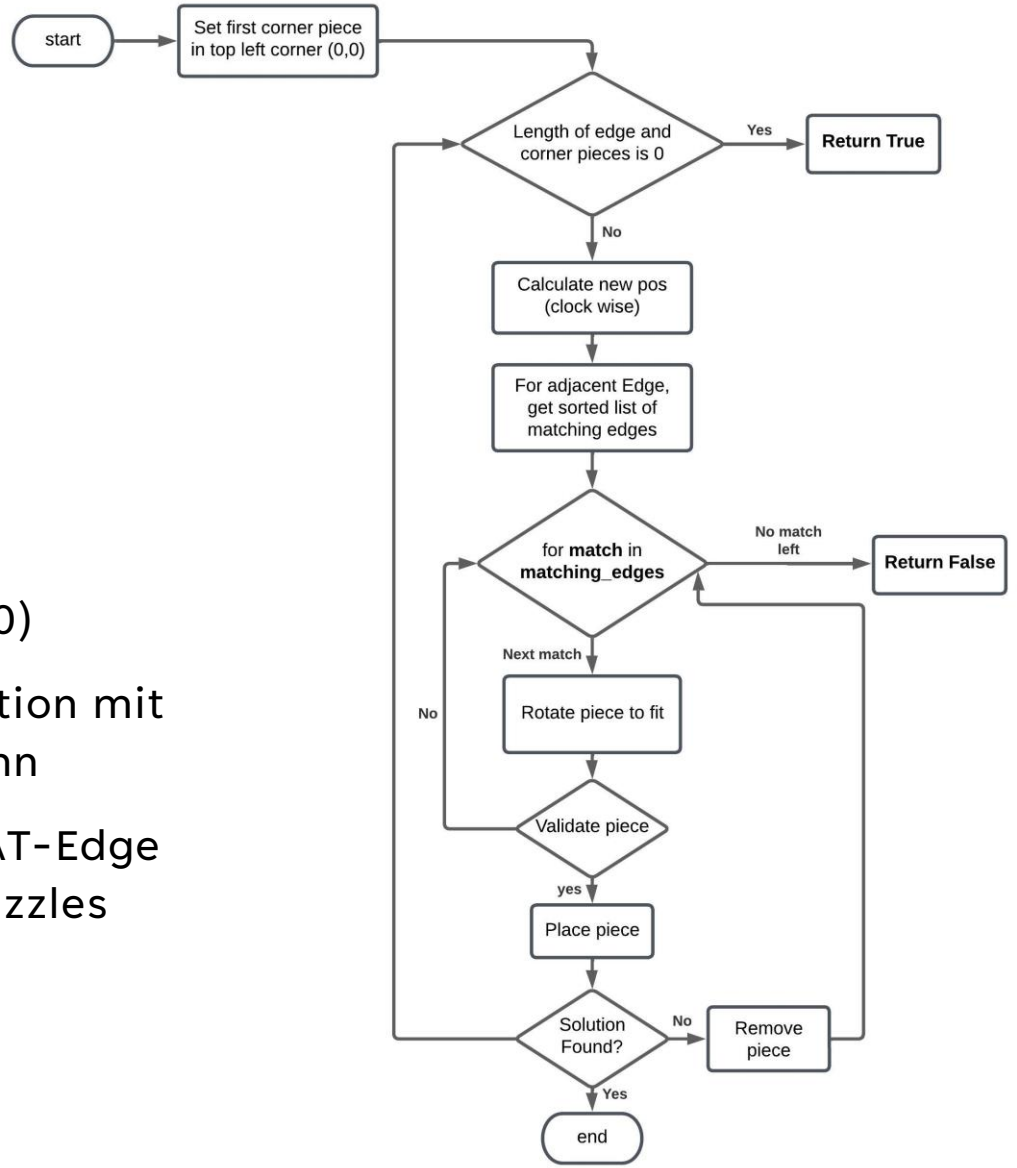


SOLVER / ALGORITHMEN /
FRAME BUILDER

- Teilproblem ist leichter zu lösen
- Betrachtung von CORNER- und EDGE-Pieces
- Löst das Dimensionsproblem

SOLVER / ALGORITHMEN / FRAME BUILDER

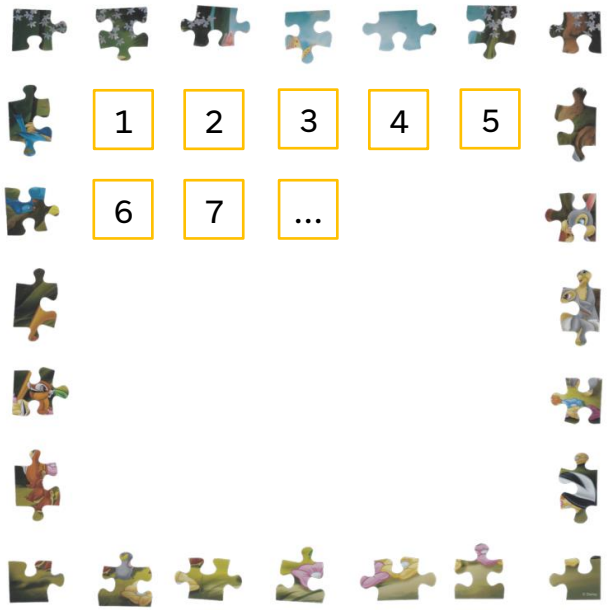
- Rekursiver Aufbau
- Initialisierung mit Ecke bei (0,0)
- Berechnung der nächsten Position mit Build Direction im Uhrzeigersinn
- Validierung möglich durch FLAT-Edge und Größeninformation des Puzzles



SOLVER / ALGORITHMEN / TIEFENSUCHE

Algorithm 3 Depth-Search-Algorithmus

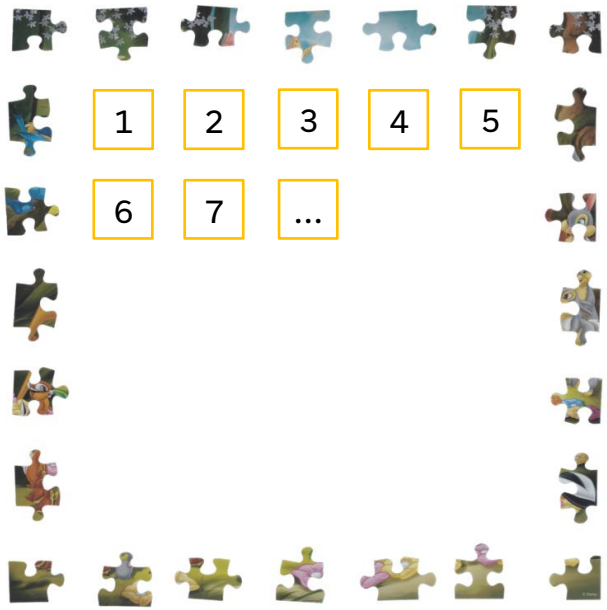
```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  start_pos ← (1,1)
4:  NEXT_PIECE(start_pos)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(pos)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                    ▷ Abbruchbedingung
9:          return True
10:     end if
11:     adjacent_edges ← puzzle.get_adjacent_edges(pos)
12:     matching_pieces ← matcher.get_matching_pieces(adjacent_edges)
13:     for matching_piece ∈ matching_pieces do
14:         Rotiere matching_piece
15:         Platziere matching_piece an pos
16:         new_pos ← get_next_position(pos)      ▷ Berechnet Position Reihe für Reihe
17:         if NEXT_PIECE(new_pos) then
18:             return True
19:         else
20:             Entferne matching_piece an pos
21:         end if
22:     end for
23: end function
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

Algorithm 3 Depth-Search-Algorithmus

```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  start_pos ← (1,1)
4:  NEXT_PIECE(start_pos)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(pos)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                    ▷ Abbruchbedingung
9:          return True
10:     end if
11:     adjacent_edges ← puzzle.get_adjacent_edges(pos)
12:     matching_pieces ← matcher.get_matching_pieces(adjacent_edges)
13:     for matching_piece ∈ matching_pieces do
14:         Rotiere matching_piece
15:         Platziere matching_piece an pos
16:         new_pos ← get_next_position(pos)      ▷ Berechnet Position Reihe für Reihe
17:         if NEXT_PIECE(new_pos) then
18:             return True
19:         else
20:             Entferne matching_piece an pos
21:         end if
22:     end for
23: end function
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

- *adjacent_edges* = puzzle.get_adjacent_edges((1, 1))

```
adjacent_edges = [Edge(34), None, None, Edge(16)]
```

- *matching_pieces* = matcher.get_matching_pieces(*adjacent_edges*)

1. Erstelle Best Match Listen

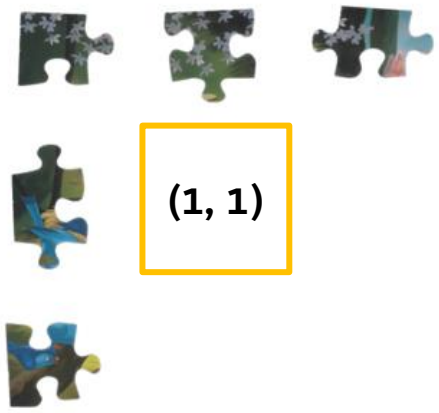
```
edge_34 = [2,8,14, … ]
edge_16 = [0,16,56,… ]
```

2. Berechne PieceID →

```
edge_34 = [0,2,3,… ]
edge_16 = [0,4,14,…]
```

3. Schnittmenge an möglichen Puzzleteilen

```
possible_pieces = [0, ..]
```



SOLVER / ALGORITHMEN / TIEFENSUCHE

4. Berechnung möglicher Piece Matches

```
possible_pieces = [0, …]
```



(0, DEG_0, 0.53)



(0, DEG_180, 0.98)

```
matching_pieces = [(0, DEG_180, 0.98), (0, DEG_0, 0.53), …]
```

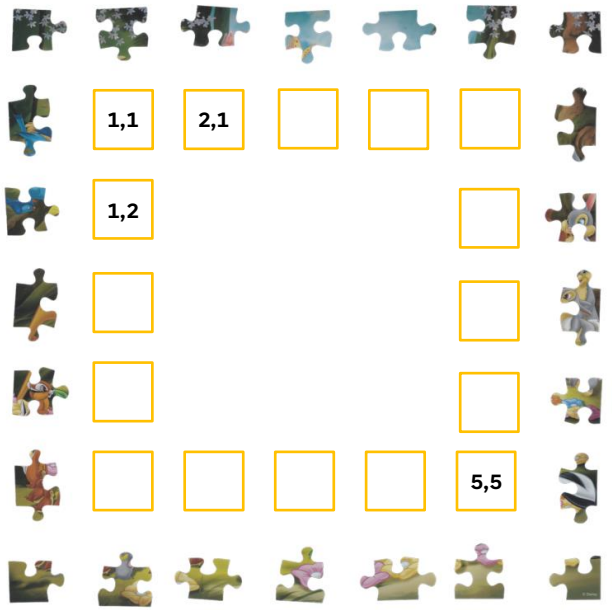


SOLVER / ALGORITHMEN / BEST-FIRST

SOLVER / ALGORITHMEN / BEST-FIRST

Algorithm 4 Best-First-Algorithmus

```
1:  Eingabe: Puzzle mit gelöstem Rahmen
2:  Ausgabe: gelöstes Puzzle
3:  exposed_edges ← Initialisiere mit puzzle          ▷ Alle inneren Kanten des Rahmens
4:  NEXT_PIECE(exposed_edges)
5:                                  ▷ Rekursive Funktion für jedes weitere Puzzleteil
6:  function NEXT_PIECE(exposed_edges)
7:      available_pieces ← puzzle.available_pieces
8:      if len(available_pieces) == 0 then                ▷ Abbruchbedingung
9:          return True
10:     end if
11:     exposed_edge_scores ← get_exposed_edge_scores()
12:     adjacent_pos_scores ← get_best_adjacent_pos(exposed_edge_scores)
13:     for pos ∈ adjacent_pos_scores do
14:         adjacent_edges ← puzzle.get_adjacent_edges(pos)
15:         matching_piece ← matcher.get_matching_pieces(adjacent_edges)[0]
16:         Rotiere matching_piece
17:         Platziere matching_piece an pos
18:         updated_exposed_edges ← Aktualisiere exposed_edges Liste
19:         if NEXT_PIECE(updated_exposed_edges) then
20:             return True
21:         else
22:             Entferne matching_piece an pos
23:         end if
24:     end for
25: end function
```



SOLVER / ALGORITHMEN / BEST-FIRST

- *exposed_edge_scores* = get_exposed_edge_scores()

```
ExEdge = (EID, ADJACENT_POS)

exposed_edge_scores = [(ExEdge(3, (1,1)), 0.97), (ExEdge(34, (3,1)), 0.95),
                       (ExEdge(8, (1,1)), 0.78), …]
```

- *adjacent_pos_scores* = get_best_adjacent_pos(*exposed_edge_scores*)

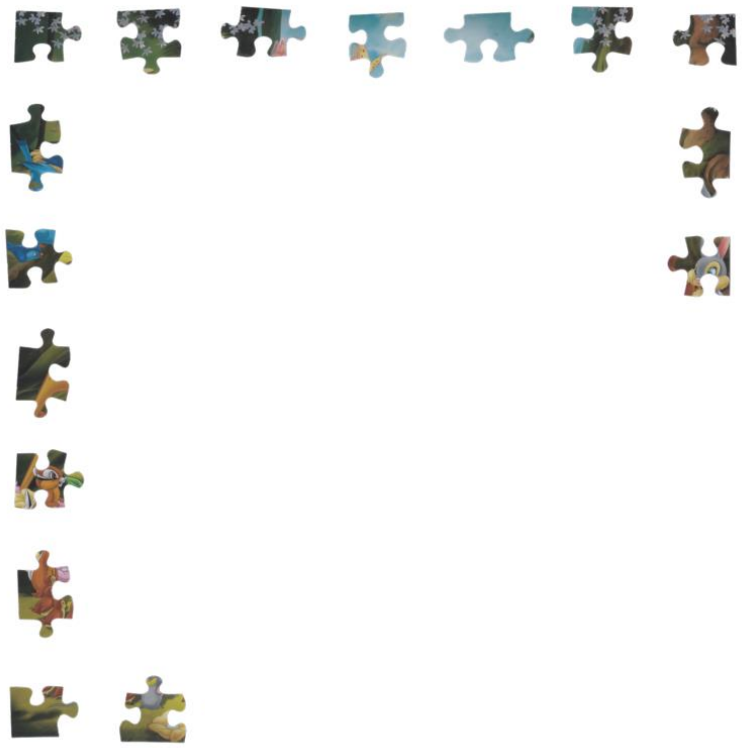
  Summiere Score pro angrenzender Position

```
adjacent_pos_scores = [((1,1), 1.75), ((3,1), 0.95) ..]
```

SOLVER / ALGORITHMEN / EASY-FIRST

- Ähnlich zu Best-First Algorithmus
- Anstatt die Möglichen Edges nach dem Besten Score zu sortieren, werden sie nach der Differenz zum nächsten Match sortiert

```
score_diff = matches[0].score - matches[1].score
```



SOLVER / ALGORITHMEN /
OPTIMIERTER FRAME BUILDER

- Ähnlich zu Best-First Algorithmus
- Übergabe von Exposed Edges anstatt Position
- Bester / Leichtester Match kann zwischen zwei statt nur einer Position gewählt werden

VIELEN DANK FÜR EURE AUFMERKSAMKEIT!

Jonas Leitner

@joleitner (gitlab)